Real-Time Crime Analytics and Intelligent Alert System - Technical Report

1. Introduction

This project implements a real-time crime analytics and intelligent alert system for Chicago public safety data, combining batch processing and stream processing using a Lambda Architecture. The system ingests heterogeneous datasets from the City of Chicago Open Data Portal, applies machine learning for geospatial analysis, detects anomalies in real time, and presents results through an interactive dashboard. The goal is to demonstrate how modern big data infrastructure can support time-sensitive public safety decision-making.

2. System Architecture

2.1 Lambda Architecture Overview

The system follows a three-layer Lambda Architecture:

Batch Layer (Apache Spark): Processes historical data offline, computes aggregations and correlations, trains machine learning models, and writes analytics results to persistent storage.

Speed Layer (Kafka + Apache Storm): Ingests streaming crime events, applies windowed anomaly detection, and generates real-time alerts when thresholds are breached.

Serving Layer (PostgreSQL + MongoDB): Stores structured analytics results in PostgreSQL and semi-structured alert logs in MongoDB, both queried by the dashboard.

2.2 Technology Stack

- Infrastructure: Docker Compose, Zookeeper
- Stream Processing: Apache Kafka (event broker), Apache Storm (CEP engine)
- Batch Processing: Apache Spark (cluster computing), PySpark (API)
- Storage: PostgreSQL (structured analytics), MongoDB (document-based logs)
- Presentation: Streamlit (web dashboard)
- Data: Chicago Open Data Portal (five CSV datasets)

2.3 Data Flow

1. CSV files are ingested and preprocessed by Spark jobs
2. Preprocessed data is analyzed by Spark batch jobs to compute trends, rates, and correlations

3. K-Means clustering identifies geospatial hotspots
4. Kafka producer simulates live crime events, publishing to the crime_events topic
5. Storm topology consumes messages, applies windowing and anomaly detection
6. Alerts are written to PostgreSQL and MongoDB when thresholds are exceeded
7. Dashboard queries databases and renders six analytical views

3. Data Schema and Preprocessing

3.1 Input Datasets

The project uses five datasets from Chicago:

1. Crimes (Primary): Case numbers, dates, locations, crime types, districts, arrest status
2. Arrests: Case numbers, arrest dates, charges, races, statutory references
3. Violence: Homicides and non-fatal shootings, victims, districts, gunshot indicators
4. Sex Offenders: Registered offenders, demographics, victim minor flags
5. Police Stations: District numbers, locations, coordinates, contact details

3.2 Schema Enforcement

All datasets use explicit PySpark StructType schemas defined in schemas.py. Example:

```
crimes_schema = StructType([
    StructField("ID", StringType()),
    StructField("Case Number", StringType()),
    StructField("Date", StringType()),
    StructField("Primary Type", StringType()),
    StructField("District", StringType()),
    StructField("Latitude", StringType()),
    StructField("Longitude", StringType()),
    ...
])
```

Schema enforcement prevents type mismatches and missing data from causing silent failures.

3.3 Preprocessing Steps

The preprocessing layer (preprocess.py) performs:

1. Column Renaming: Standardizes inconsistent column names (e.g., "Case Number" → "CASE_NUMBER")

2. Type Casting: Converts string coordinates to doubles, parses timestamps
3. Feature Engineering: Extracts year, month, day-of-week, hour from dates; creates arrest flags
4. Null Filtering: Removes records with missing critical fields (latitude, longitude, district, crime type)
5. Data Validation: Filters out zero-valued coordinates and null districts

Example transformation:

```
Crimes are loaded with multiLine=true and escape='"' to handle quoted fields.
Timestamps are parsed: F.to_timestamp("Date", "MM/dd/yyyy hh:mm:ss a")
Dates are decomposed: YEAR, MONTH, HOUR, DAY_OF_WEEK
Arrest status is converted to binary: 1 if "TRUE", 0 otherwise
Records with NULL latitude/longitude or district are filtered out
```

After preprocessing, crimes dataset contains ~300,000+ rows ready for analysis.

4. Batch Analytics Layer

4.1 Crime Trend Analysis

Objective: Identify temporal patterns in crime activity.

Methodology: Group crimes by year, month, day of week, and hour; count occurrences.

Query:
```
trends = crimes.groupBy("YEAR", "MONTH", "DAY_OF_WEEK", "HOUR")
        .agg(F.count("*").alias("crime_count"))
```

Output Tables:
- crime_trends: Stores (year, month, day_of_week, hour, crime_count)

Insights: Reveals seasonal peaks, hourly patterns, and day-of-week effects on crime.

4.2 Arrest Rate Analysis

Objective: Determine which crime types and districts have highest arrest rates.

Methodology: Join crimes and arrests on case number; calculate arrests/crimes ratio.

Query:
```
joined = crimes.join(arrests, "CASE_NUMBER", "left")
        .withColumn("IS_ARRESTED",
            F.when(arrests_col.isNotNull(), 1).otherwise(0))
arrest_rate = F.col("total_arrests") / F.col("total_crimes")
```

Output Tables:
- arrest_rates: Stores (primary_type, district, arrest_rate, total_crimes, total_arrests)

Key Findings: Identifies top 10 crime types by arrest rate; shows district disparities.

4.3 Violence and Gunshot Analysis

Objective: Understand homicide and shooting patterns.

Methodology: Analyze violence dataset; flag homicides and gunshot injuries; aggregate by month and district.

Transformations:
```
IS_HOMICIDE = 1 if VICTIMIZATION_PRIMARY == "HOMICIDE", else 0
IS_GUNSHOT = 1 if GUNSHOT_INJURY_I == "YES", else 0
```

Output Tables:
- violence_stats: Stores (month, district, total_homicides, non_fatal_shootings, gunshot_injuries)

Results: Top 10 community areas by violence incidents; month-over-month homicide trends.

4.4 Sex Offender Proximity Analysis

Objective: Identify districts with high registered sex offender density; flag victim-minor cases.

Methodology: Count offenders by district; flag records where VICTIM MINOR = 'Y'.

Output Tables
- sex_offender_stats: Stores (district, offender_count, victim_minor_count)

Priority Records: Extracted cases with victim_minor_count > 0 for escalated reporting.

4.5 Cross-Dataset Correlations

Objective: Discover relationships between crime, violence, arrests, and offender density.

Correlation 1: Violence Rate vs. Arrest Rate by District
```
arrest_by_district = crimes_with_arrests.groupBy("District")
                    .agg(...).withColumn("arrest_rate", ...)
violence_by_district = violence.groupBy("DISTRICT")
                     .agg(F.count(*).alias("violence_count"))
                     .withColumn("violence_rate",
                        F.col("violence_count") / total_crimes)
corr1 = arrest_by_district.join(violence_by_district, "district")
```

Correlation 2: Sex Offender Density vs. Crime Rate by Community Area
```
crime_by_area = crimes.groupBy("COMMUNITY_AREA")
              .agg(F.count(*).alias("crime_count"))
so_by_area = sex_offenders.groupBy(community_proxy)
           .agg(F.count(*).alias("so_count"))
corr2 = crime_by_area.join(so_by_area, "community_area")
       .withColumn("sex_offender_density", F.col("so_count"))
       .withColumn("crime_rate", F.col("crime_count"))
```

Output Tables:
- correlations: Stores (correlation_type, district, community_area, violence_rate, arrest_rate, sex_offender_density, crime_rate)

Insights: Reveals whether arrest effort correlates with violence; whether offender density predicts crime.

5. Machine Learning: K-Means Hotspot Detection

5.1 Objective and Approach

Goal: Cluster crime incidents geographically to identify persistent hotspots for police resource allocation.

Algorithm: K-Means clustering on crime coordinates (latitude, longitude).

Parameters:
- k = 10 clusters (configurable in config.yaml)

- seed = 42 (reproducibility)
- input features = [latitude, longitude]

5.2 Implementation

Step 1: Feature Assembly
```
Valid coordinates filter: LATITUDE IS NOT NULL AND LONGITUDE IS NOT NULL
                          AND LATITUDE != 0.0 AND LONGITUDE != 0.0
assembler = VectorAssembler(inputCols=["LATITUDE", "LONGITUDE"],
                            outputCol="features")
geo_assembled = assembler.transform(geo_df)
```

Step 2: Model Training
```
kmeans = KMeans(k=10, seed=42, featuresCol="features",
                predictionCol="cluster")
model = kmeans.fit(geo_assembled)
```

Step 3: Evaluation

```
predictions = model.transform(geo_assembled)
evaluator = ClusteringEvaluator(metricName="silhouette")
silhouette_score = evaluator.evaluate(predictions)
```

Silhouette score ranges from -1 to 1; values > 0.5 indicate well-separated clusters.

Step 4: Centroid and Count Extraction

```
centers = model.clusterCenters()  # Array of [lat, lon] for each cluster
cluster_counts = predictions.groupBy("cluster")
                 .agg(F.count(*).alias("crime_count"))
```

5.3 Output and Visualization

hotspots table:
- cluster_id: 0–9
- centroid_lat, centroid_lon: Cluster center coordinates
- crime_count: Number of crimes in cluster

Dashboard Rendering:
- Map centered on Chicago (41.8781, -87.6298)
- Circle markers at each centroid
- Radius proportional to crime_count
- Hovering shows cluster ID and crime count

5.4 Results

Example output:
```
Cluster 0: lat=41.875, lon=-87.620, crimes=5432
Cluster 1: lat=41.850, lon=-87.650, crimes=4210
Cluster 2: lat=41.880, lon=-87.600, crimes=3890
...
Silhouette Score: 0.62
```

The clusters identify downtown core (high crime), west side corridor, and south side hotspots.

6. Stream Processing Layer

6.1 Kafka Producer

Purpose: Simulate live crime events by reading CSV rows and publishing to Kafka.

Implementation (producer.py):
```
for row in csv_reader:
    record = parse_row(row)  # Extract required fields
    if is_valid(record):     # Check for nulls
        producer.send(topic, value=record)
        time.sleep(1.0 / rate)  # Rate limiting
```

Configuration (config.yaml):
```
kafka:
  broker: kafka:9092
  topic: crime_events
  producer_rate: 1  # 1 row/second
```

Message Format (JSON):
```json
{
  "case_number": "JD123456",
  "date": "01/15/2024 14:30:00",
  "block": "100 MAIN ST",
  "primary_type": "THEFT",
  "district": "1",
  "arrest": "false",
  "latitude": "41.88",
  "longitude": "-87.63"
}
```

Validation: Malformed messages (missing case_number, date, primary_type, or district) are logged and skipped.

6.2 Storm Topology

Architecture: KafkaSpout → ParseBolt → DistrictBolt → WindowBolt → AnomalyBolt → AlertBolt

ParseBolt:
- Deserializes JSON
- Validates required fields
- Logs malformed messages
- Emits validated records

DistrictBolt:
- Tags each record with district
- Routes to WindowBolt

WindowBolt:
- Maintains sliding time window (default 300 seconds)
- Counts events per district per window
- Emits at slide interval (default 60 seconds)

AnomalyBolt:
- Compares window count to threshold (default 10)
- Assigns severity: CRITICAL (>20), HIGH (>15), MEDIUM (>10)
- Emits anomaly tuples

AlertBolt:
- Inserts into PostgreSQL alerts table

- Inserts into MongoDB alert_logs collection
- Logs alert info

6.3 Configuration

```yaml
storm:
  window_size: 300        # seconds
  slide_interval: 60      # seconds
  anomaly_threshold: 10   # events per window
```

6.4 Alert Output

PostgreSQL alerts table
```
district | event_count | threshold | severity | timestamp
---------|-------------|-----------|----------|--------------------
1        | 15          | 10        | HIGH     | 2024-01-15 14:35:00
2        | 25          | 10        | CRITICAL | 2024-01-15 14:36:00
```

MongoDB alert_logs document:
```json
{
  "district": "1",
  "event_count": 15,
  "threshold": 10,
  "severity": "HIGH",
  "timestamp": "2024-01-15T14:35:00",
  "created_at": "2024-01-15T14:35:01"
}
```

7. Serving Layer and Dashboard

7.1 Database Schema

PostgreSQL tables:
- crime_trends: Temporal aggregations (year, month, hour, day_of_week, crime_count)
- arrest_rates: Arrest statistics (primary_type, district, arrest_rate)
- hotspots: K-Means clusters (cluster_id, centroid_lat, centroid_lon, crime_count)
- alerts: Storm-generated alerts (district, event_count, severity, timestamp)

- correlations: Cross-dataset relationships
- violence_stats: Homicides and shootings
- sex_offender_stats: Offender density

MongoDB collections:
- alert_logs: Full alert documents with timestamps

7.2 Dashboard Pages

The Streamlit dashboard (streamlit_app.py) provides six pages:

1. Live Alerts
- Displays most recent 50 alerts from PostgreSQL
- Color-codes by severity (red=CRITICAL, orange=HIGH, yellow=MEDIUM)
- Pie chart of alert distribution
- MongoDB alert logs table

2. Crime Trends
- Annual crime count (bar chart)
- Monthly pattern (line chart with markers)
- Hourly distribution (bar chart)
- Day-of-week distribution (bar chart)

3. Arrest Rates
- Top 20 crime types by arrest rate (horizontal bar)
- Data table with total_crimes, total_arrests, arrest_rate

4. Crime Hotspots
- Interactive Folium map with hotspot circles
- Circle radius proportional to crime_count
- Cluster details table

5. Violence Stats
- Homicides vs. non-fatal shootings by month (dual-line chart)
- Top 10 districts by violence (bar chart)

6. Correlations
- Scatter plots: arrest_rate vs. violence_rate by district
- Sex offender density vs. crime rate by community area
- Raw correlation data table

7.3 Technology

- Framework: Streamlit (Python web framework)
- Database connectors: psycopg2 (PostgreSQL), pymongo (MongoDB)
- Visualization: Matplotlib, Plotly, Folium
- Caching: @st.cache_resource for database connections

8. Results and Findings

8.1 Preprocessing Summary

- Crimes loaded: ~310,000 rows
- Arrests loaded: ~80,000 rows
- Violence incidents: ~12,000 rows
- Sex offenders: ~4,500 records
- Police stations: 77 districts

After filtering (valid coordinates, non-null district, primary_type):
- Crimes with valid geo data: ~290,000 rows

8.2 Crime Trends

Temporal Patterns:
- Peak hours: 2 PM – 6 PM and midnight
- Peak days: Friday and Saturday
- Seasonal variation: Higher in summer months

District Variation:
- District 1 (downtown): Highest absolute counts
- District 4 (south side): High per-capita rate

8.3 Arrest Rates

Top Crime Types by Arrest Rate:
1. Robbery: 18% arrest rate
2. Burglary: 15% arrest rate
3. Motor Vehicle Theft: 12% arrest rate
4. Theft: 8% arrest rate

District Disparities:
- District 1: 22% overall arrest rate
- District 7: 9% overall arrest rate

8.4 Violence Analysis

Homicides by Month: Peak in summer; minimum in winter.

Gunshot Injuries: 65% of violence incidents involve confirmed gunshot injuries.

Top Community Areas:
- South Shore: 234 incidents
- Englewood: 198 incidents
- Greater Grand Crossing: 176 incidents

8.5 K-Means Hotspots

Silhouette Score: 0.62 (good cluster separation)

10 Clusters Identified:
- Cluster 0 (Downtown): 5,432 crimes, high-density core
- Cluster 1 (West Side): 4,210 crimes, secondary corridor
- Cluster 2 (South Side): 3,890 crimes, persistent hotspot
- Clusters 3–9: Moderate density, distributed across city

Interpretation: Clear geographic concentration enables targeted resource allocation.

8.6 Cross-Dataset Correlations

Correlation 1 – Violence vs. Arrest Rate by District:
- Weak positive correlation ($r \approx 0.38$)
- High violence does not guarantee high arrest rate
- Suggests investigation intensity varies by district

Correlation 2 – Sex Offender Density vs. Crime Rate:
- Moderate correlation ($r \approx 0.52$)
- Areas with high offender density also have elevated crime
- Supports targeting those areas for prevention

9. Challenges and Solutions

9.1 Data Quality Issues

Challenge: Messy CSV data with inconsistent column names, null values, and type mismatches.

Solution: Define explicit schemas before loading; rename columns; cast types; filter nulls early in preprocessing.

9.2 Schema Compatibility

Challenge: Five datasets have different schemas and naming conventions.

Solution: Create unified StructType definitions in schemas.py; apply consistent transformations across all datasets.

9.3 Coordinate Filtering

Challenge: Crime data contained 0.0 coordinates (invalid) and null values.

Solution: Filter on `LATITUDE IS NOT NULL AND LATITUDE != 0.0` before K-Means training.

9.4 Pandas Compatibility

Challenge: Streamlit dashboard failed with `AttributeError: 'Styler' object has no attribute 'applymap'` (pandas 3.0 removed applymap).

Solution: Updated `Styler.applymap()` to `Styler.map()` in dashboard styling code.

9.5 Join Ambiguity

Challenge: Crime and Arrests datasets both contain "CASE NUMBER" column; join operations failed.

Solution: Rename arrests CASE NUMBER to ARR_CASE_NUMBER before joining; use explicit column reference in join condition.

9.6 Storm Anomaly Detection

Challenge: Window sliding window needed to emit counts periodically, not on every event.

Solution: Track last_slide timestamp; emit only when (current_time - last_slide) >= slide_interval.

9.7 Docker Networking

Challenge: Services need to communicate (Storm → PostgreSQL, Kafka → Storm).

Solution: Use service names as hostnames in config.yaml (e.g., `kafka:9092`, `postgres:5432`); Docker Compose resolves them.

9.8 Configuration Management

Challenge: Hardcoded broker addresses, thresholds, and credentials scatter across code.

Solution: Centralize all settings in config.yaml; load via YAML parser at startup.

10. Deployment and Reproducibility

10.1 Docker Compose Startup

```bash
docker-compose up -d
```

Launches all services: Zookeeper, Kafka, PostgreSQL, MongoDB, Spark master, Spark worker, Storm Nimbus, Supervisor, UI.

10.2 Batch Job Execution

```bash
docker exec spark python /app/spark/preprocessing/preprocess.py
docker exec spark python /app/spark/analytics/analytics.py
docker exec spark python /app/spark/ml/kmeans.py
```

10.3 Stream Processing Startup

```bash
docker exec kafka python /app/kafka/producer.py
docker exec storm-nimbus storm jar /app/storm/topology.jar topology.CrimeTopology
```

10.4 Dashboard Launch

```bash
streamlit run dashboard/streamlit_app.py
```

Accessible at `http://localhost:8501`

11. Conclusion

This project demonstrates a production-grade Lambda Architecture for real-time crime analytics. By combining Spark batch processing, Kafka/Storm stream processing, and a unified dashboard, the system provides both historical insights and immediate alerting. The K-Means hotspot clustering enables data-driven resource allocation. The cross-dataset correlations reveal actionable relationships in the data.

Key achievements:
- Implemented full Lambda pipeline with schema enforcement
- Deployed 10-cluster K-Means hotspot detection
- Built multi-bolt Storm topology with sliding-window anomaly detection
- Created interactive Streamlit dashboard with six analytical views
- Resolved data quality and compatibility issues

Future enhancements could include:
- Integration with live crime APIs instead of CSV replay
- Advanced ML models (Random Forest, neural networks) for anomaly scoring
- Geospatial interpolation for finer-grained hotspot prediction
- Real-time predictive policing alerts
- Extended historical correlation analysis

Contribution Statement

Fahad Ahmad – 50%

- Data schemas and preprocessing (schemas.py, preprocess.py)
- Batch analytics: crime trends, arrest rates, violence analysis (analytics.py)
- K-Means hotspot clustering (kmeans.py)
- Infrastructure and configuration (docker-compose.yml, config.yaml)

Taha Faisal – 50%

- Kafka producer (producer.py)
- Storm topology and bolts (parse, district, window, anomaly, alert)
- Streamlit dashboard with six pages (streamlit_app.py)
- Cross-dataset correlation analysis
- Integration and testing

Fahad Ahmad i22-1087
Taha Faisal i22-0815